WEEK-2 Practical

TASk-1 Develop a C program that uses the system calls open(), read(), write(), and closeo to copy the contents of one file to 3 another. Explain how control transitions between user Space and kernel space during execution

saicharan_reddi@SAICHARAN

GNU nano 8.7.1 copyfile.c *

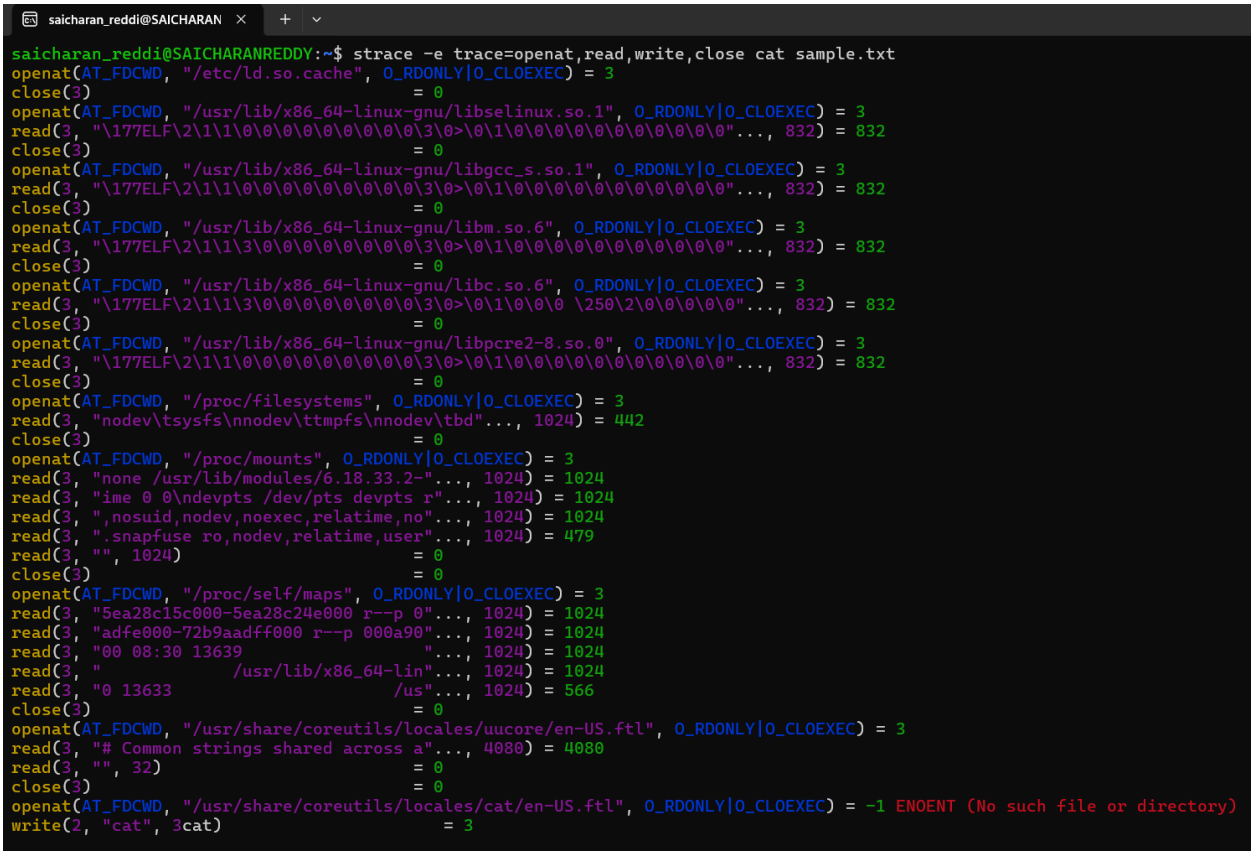
```c
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int source, destination;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytes_read = read(source, buffer, BUFFER_SIZE)) > 0)
```

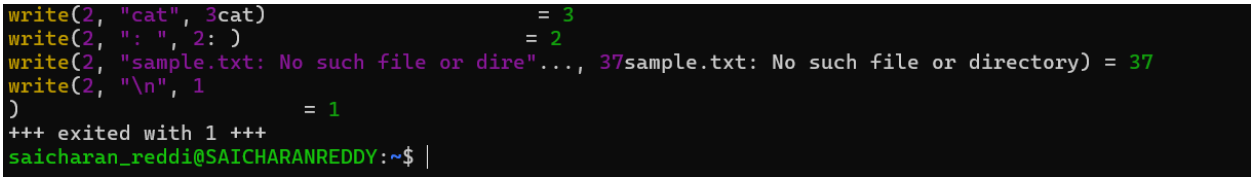
```
saicharan_reddi@SAICHARANREDDY:~$ nano filecopy.c
saicharan_reddi@SAICHARANREDDY:~$ gcc -Wall -Wextra filecopy.c -o bin/filecopy
saicharan_reddi@SAICHARANREDDY:~$ echo "This is a sample file for system call testing." > sample.txt
saicharan_reddi@SAICHARANREDDY:~$ ./bin/filecopy sample.txt copy.txt
File copied successfully.
saicharan_reddi@SAICHARANREDDY:~$ cat sample.txt
This is a sample file for system call testing.
saicharan_reddi@SAICHARANREDDY:~$ cat copy.txt
This is a sample file for system call testing.
```

Report: I developed a C program to copy the contents of one file to another using the system calls open(), read(), write(), and close(). The source file was opened in read-only mode and the destination file was created for writing. The contents were read into a buffer and written to the

destination file. During execution, the system calls transfer control from user space to kernel space, where the kernel performs the required file operations and returns the results to the program. The file was copied successfully.

Task-2 : Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system 3 calls and identify the kernel services involved





Report: I used the strace utility to trace the system calls generated by the command cat sample.txt. The trace showed important system calls such as openat(), read(), write(), and close(). openat() opens the file, read() obtains the file contents, write() displays the contents on the terminal, and close() closes the file descriptor. The experiment helped me understand how a user-level command interacts with kernel services through system calls.